T11 Árboles, bosques, bagging y boosting

Índice

1. Introducción
2. Árboles
3. Bagging
4. Bosques
5. Boosting
6. Gradient boosting
7. Interpretación

1 Introducción

Modelos paramétricos: **usan los datos solo en entrenamiento** para ajustar los parámetros que los gobiernan

Modelos no paramétricos: **mantienen los datos en inferencia** ya que, en esencia, sus parámetros son los propios datos

Principal dificultad de los no paramétricos: elevado coste computacional en inferencia con muchos datos

Técnicas no paramétricas: tres tipos (según PML1)

- **Basadas en ejemplos:** K-vecinos más próximos y estimación con kernels densidad
- **Métodos kernel:** procesos Gaussianos y máquinas de vectores soporte
- **Árboles, bosques, bagging y boosting:** ensambles de estructuras jerárquicas para clasificación y regresión

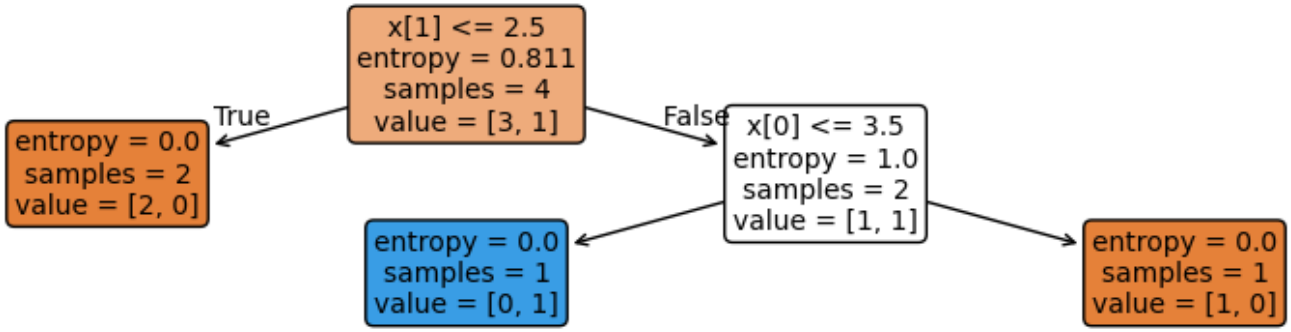
Gradient (tree) boosting: técnica competitiva con las redes para datos tabulados; librería **XGBoost**

Objetivo: introducir brevemente árboles, bosques, bagging y boosting

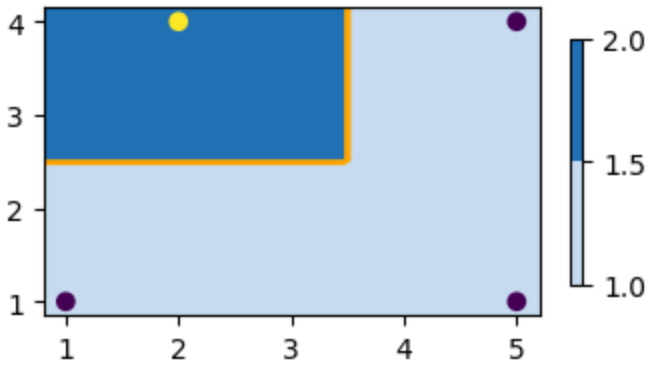
2 Árboles

Ejemplo de árbol para clasificación: aprendido con $\mathcal{D} = \{((1,1)^t, 1), ((2,4)^t, 2), ((5,1)^t, 1), ((5,4)^t, 1)\}$

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; from sklearn.tree import DecisionTreeClassifier, plot_tree
X = np.array(([1, 1], [2, 4], [5, 1], [5, 4])); y = np.array([1, 2, 1, 1])
dt = DecisionTreeClassifier(criterion='entropy').fit(X, y)
plt.figure(figsize=(12, 2)); plot_tree(dt, filled=True, rounded=True, fontsize=10);
```



In [ ]:
```
_, ax = plt.subplots(figsize=(4, 2)); ax.scatter(*X.T, c=y)
xx, yy = np.meshgrid(np.linspace(*ax.get_xlim(), 100), np.linspace(*ax.get_ylim(), 100))
zz = dt.predict(np.c_[xx.ravel(), yy.ravel()]); ax.contour(xx, yy, zz.reshape(xx.shape), 1, colors='orange')
cp = ax.contourf(xx, yy, zz.reshape(xx.shape), 1, cmap='Blues'); plt.colorbar(cp, ax=ax, shrink=0.8)
ax.scatter(*X.T, c=y);
```



Classification and regression trees (CART): árboles que particionan el espacio de entrada recursivamente hasta alcanzar las hojas, cada una de ellas caracterizada por la región en la que se aplica y su (modelo de) predicción correspondiente

- Dado $\boldsymbol{x} \in \mathbb{R}^D$, un árbol puede verse como un conjunto de reglas de decisión anidadas hasta alcanzar las hojas
- Cada regla de decisión o **nodo interno** i define un **split paralelo a un eje:** compara una característica d_i de la entrada con un umbral t_i y, si $\boldsymbol{x}_{d_i} \leq t_i$, $\boldsymbol{x}$ se sigue procesando por la rama izquierda; si no, se procesa por la derecha
- El procesamiento de $\boldsymbol{x}$ termina al llegar a un nodo hoja, donde se especifica la salida predicha para toda entrada dentro de su región asociada, esto es, la región acorde con los splits definidos en sus nodos antecesores

Árbol de regresión: $\boldsymbol{\theta} = \{(R_j, w_j) : j = 1 : J\}$, de J hojas, donde R_j y w_j son la región y salida de la hoja j

$$f(\boldsymbol{x}; \boldsymbol{\theta}) = \sum\nolimits_j w_j \, \mathbb{I}(\boldsymbol{x} \in R_j)$$

Árbol de clasificación: las hojas contienen una distribución sobre las etiquetas de clase

Ajuste de un árbol de J hojas: minimización en $\boldsymbol{\theta} = \{(R_j, w_j) : j = 1 : J\}$ de la pérdida empírica

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_n \ell(y_n, f(\boldsymbol{x}; \boldsymbol{\theta})) = \sum_j \sum_{\boldsymbol{x}_n \in R_j} \ell(y_n, w_j)$$

Soluciones aproximadas: métodos voraces como **CART, C4.5 o ID3,** que añaden un nodo en cada iteración

Splits de un nodo i con una característica j real: consideramos un split de $\mathcal{D}_i = \{(\boldsymbol{x}_n, y_n)\}$ por cada $t \in \{x_{nj}\}$

$$\mathcal{D}_i^L(j,t) = \{(\boldsymbol{x}_n, y_n) \in \mathcal{D}_i : x_{nj} \leq t\} \quad \text{y} \quad \mathcal{D}_i^R(j,t) = \{(\boldsymbol{x}_n, y_n) \in \mathcal{D}_i : x_{nj} > t\}$$

Splits de un nodo i con una característica j categórica: consideramos un split de $\mathcal{D}_i$ por cada categoría t

$$\mathcal{D}_i^L(j,t) = \{(\boldsymbol{x}_n, y_n) \in \mathcal{D}_i : x_{nj} = t\} \quad \text{y} \quad \mathcal{D}_i^R(j,t) = \{(\boldsymbol{x}_n, y_n) \in \mathcal{D}_i : x_{nj} \neq t\}$$

Coste o impureza de un nodo: definimos una función $c(\cdot)$ para evaluarla, independiente del tamaño del nodo

Coste de un nodo en regresión: se suele usar el **error cuadrático medio** (con respecto a la respuesta media del nodo)

$$c(\mathcal{D}_i) = \frac{1}{|\mathcal{D}_i|} \sum_{(\boldsymbol{x}_n, y_n) \in \mathcal{D}_i} (y_n - \bar{y})^2 \qquad \text{con} \qquad \bar{y} = \frac{1}{|\mathcal{D}_i|} \sum_{(\boldsymbol{x}_n, y_n) \in \mathcal{D}_i} y_n$$

Coste de un nodo en clasificación: las funciones usuales se basan en la distribución empírica de las clases en el nodo

$$\hat{\pi}_{ic} = \frac{1}{|\mathcal{D}_i|} \sum_{(\boldsymbol{x}_n, y_n) \in \mathcal{D}_i} \mathbb{I}(y_n = c)$$

- **Índice Gini:** mide el error de clasificación esperado en el nodo i; esto es, la probabilidad de que un dato al azar se clasifique mal si su clase se determina aleatoriamente según las probabilidades de las clases

$$G_i = \sum_c \hat{\pi}_{ic}(1 - \hat{\pi}_{ic}) = \sum_c \hat{\pi}_{ic} - \sum_c \hat{\pi}_{ic}^2 = 1 - \sum_c \hat{\pi}_{ic}^2$$

- **Entropía:** de la distribución empírica de las clases en el nodo

$$H_i = \mathbb{H}(\hat{\boldsymbol{\pi}}_i) = -\sum_c \hat{\pi}_{ic} \log \hat{\pi}_{ic}$$

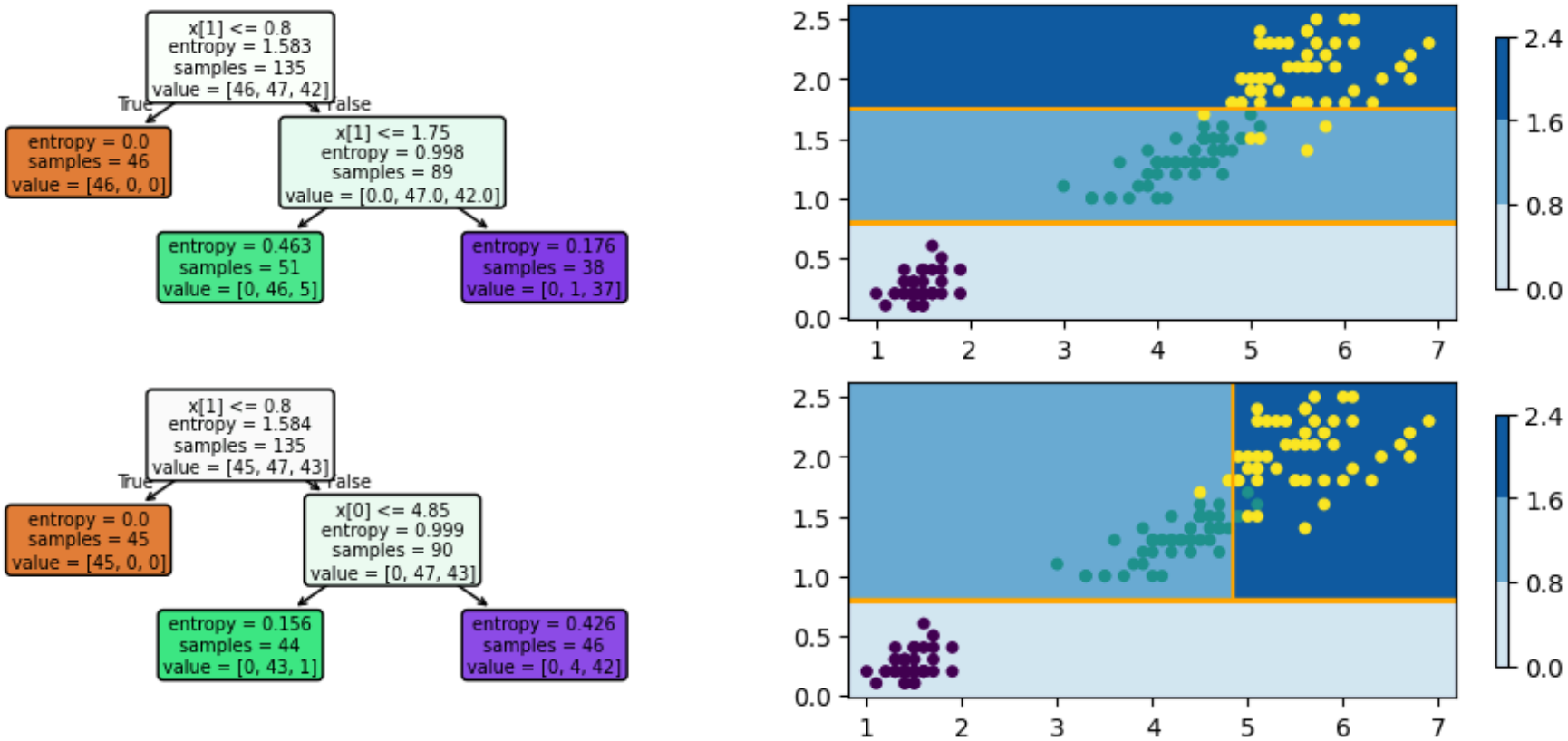
Mejor split de un nodo i: reduce al máximo su coste, esto es, minimiza la suma de los costes normalizados de los hijos

$$(j_i, t_i) = \arg \min_{j \in \{1,\ldots,D\}} \min_{t \in \mathcal{T}_j} \frac{|\mathcal{D}_i^L(j,t)|}{|\mathcal{D}_i|} c(\mathcal{D}_i^L(j,t)) + \frac{|\mathcal{D}_i^R(j,t)|}{|\mathcal{D}_i|} c(\mathcal{D}_i^R(j,t))$$

Regularización: el crecimiento suele pararse al tener pocos ejemplos en un nodo o alcanzar una profundidad máxima

Incoveniente: inestabilidad (varianza); pequeños cambios en los datos pueden producir árboles muy distintos

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split; from sklearn.tree import DecisionTreeClassifier, plot_tree
iris = load_iris(); ndx = [2, 3]; X = iris.data[:, ndx]; y = iris.target  # petal length and width
nrows = 2; fig, axes = plt.subplots(nrows, 2, figsize=(12, 2.5 * nrows))
for n in np.arange(nrows):
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, shuffle=True)
    dt = DecisionTreeClassifier(criterion='entropy', max_depth=2).fit(X_train, y_train)
    plot_tree(dt, filled=True, rounded=True, ax=axes[n, 0], fontsize=7)
    ax = axes[n, 1]; ax.scatter(*X_train.T, c=y_train)
    xx, yy = np.meshgrid(np.linspace(*ax.get_xlim(), 100), np.linspace(*ax.get_ylim(), 100))
    zz = dt.predict(np.c_[xx.ravel(), yy.ravel()]); ax.contour(xx, yy, zz.reshape(xx.shape), 2, colors='orange')
    cp = ax.contourf(xx, yy, zz.reshape(xx.shape), 2, cmap='Blues')
    plt.colorbar(cp, ax=ax, shrink=0.8); ax.scatter(*X.T, c=y, s=16);
```



3 Bagging

Ensamble: reduce la varianza promediando M modelos base $\{f_m\}$

$$f(y \mid \boldsymbol{x}) = \frac{1}{M} \sum_{m=1}^{M} f_m(y \mid \boldsymbol{x})$$

- **Regresión:** sesgo similar al de los modelos base, pero mejor precisión por la menor varianza
- **Clasificación:** la salida se decide por el **método comité,** esto es, por voto mayoritario

Bagging (bootstrap aggregating): ensambla M modelos ajustados con diferentes versiones de los datos, obtenidas por boostraping (muestreo con reemplazamiento)

Desventaja: cada modelo base ve un 63% de datos aprox.; en el límite, la probabilidad de que un dato no se seleccione es

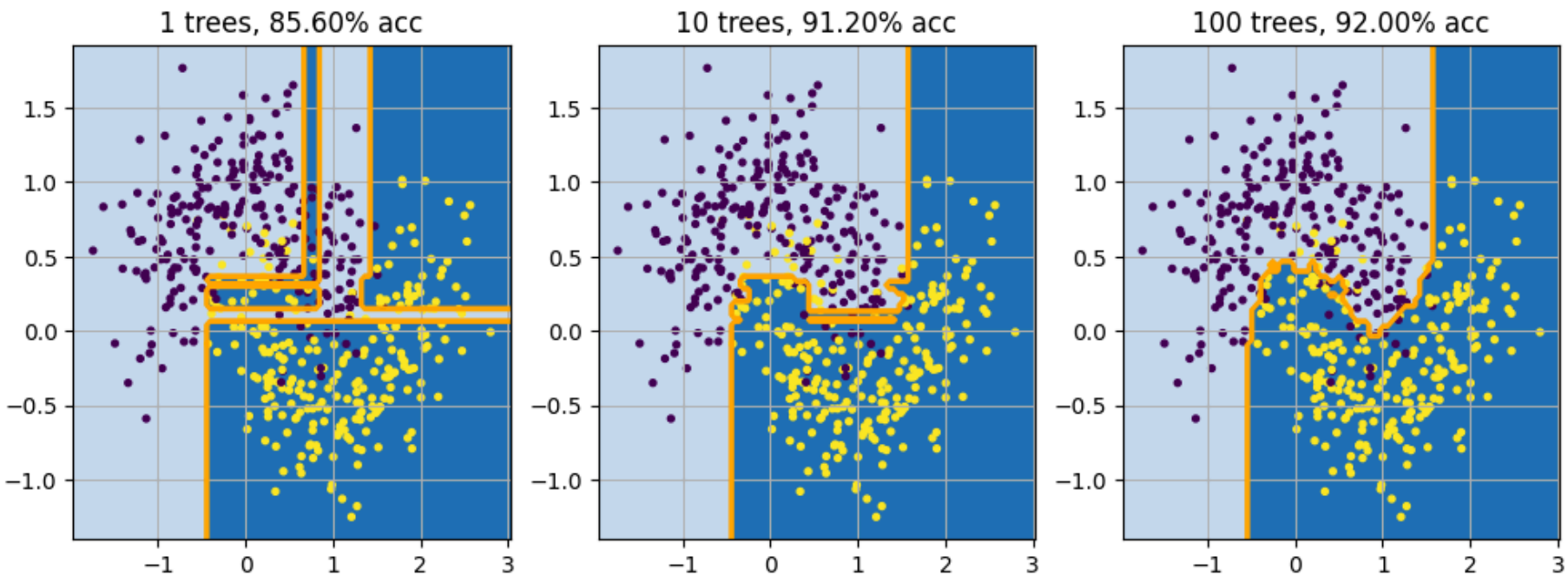
$$p = \lim_{N \to \infty} (1 - 1/N)^N = e^{-1} \approx 0.37$$

Ventaja: el 37% de muestras **out-of-bag** puede usarse en test

Ventaja principal: no depende demasiado de ningún dato individual, lo que favorece mayor robustez y generalización

Ejemplo: bagging de árboles

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split; from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier; from sklearn.metrics import accuracy_score
X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
bag_sizes = [1, 10, 100]; nrows = 1; ncols = 3
fig, axes = plt.subplots(nrows, ncols, figsize=(12, 4))
for i, bag_size in enumerate(bag_sizes):
    ax = axes.flat[i]; ax.scatter(*X.T, c=y, s=8)
    clf = BaggingClassifier(DecisionTreeClassifier(random_state=42), n_estimators=bag_size,
        max_samples=100, bootstrap=True, random_state=42).fit(X_train, y_train)
    y_pred = clf.predict(X_test); acc = accuracy_score(y_test, y_pred)
    ax.grid(); xx, yy = np.meshgrid(np.linspace(*ax.get_xlim(), 100), np.linspace(*ax.get_ylim(), 100))
    zz = clf.predict(np.c_[xx.ravel(), yy.ravel()])
    ax.contour(xx, yy, zz.reshape(xx.shape), 1, colors='orange', linestyles='solid')
    ax.contourf(xx, yy, zz.reshape(xx.shape), 1, cmap='Blues'); ax.scatter(*X.T, c=y, s=8)
    ax.set_title('{} trees, {:.2%} acc'.format(bag_size, acc))
```



Ejemplo: clasificación de correos en spam y no-spam

In [ ]:
```
import pandas as pd; import numpy as np
from sklearn.ensemble import BaggingClassifier; from sklearn.metrics import accuracy_score
data_url = 'https://github.com/empathy87/The-Elements-of-Statistical-Learning-Python-Notebooks/blob/master/data/'
file = 'Spam.txt?raw=True'; df = pd.read_csv(data_url + file)
is_test = df.test.values; y = df.spam.values; X = df.drop(['test','spam'], axis=1).to_numpy(copy=True)
X_train, X_test = X[is_test == 0], X[is_test == 1]
y_train, y_test = y[is_test == 0], y[is_test == 1]
ntrees_list = [10, 100, 1000, 10000]
for ntrees in ntrees_list:
    clf = BaggingClassifier(n_estimators=ntrees, random_state=10, bootstrap=True).fit(X_train, y_train)
    y_test_hat = clf.predict(X_test)
    acc = accuracy_score(y_test, y_test_hat)
    print(f'Bagged {ntrees} trees, test err {1 - acc:.1%}')
```

```
Bagged 10 trees, test err 5.9%
Bagged 100 trees, test err 5.4%
Bagged 1000 trees, test err 5.5%
Bagged 10000 trees, test err 5.3%
```

4 Bosques

Random forests: variante de bagging que mejora la decorrelación de modelos base con aleatorización, no solo de datos, sino también de variables de entrada; la característica de split j_i se optimiza sobre un conjunto aleatorio $S_i \subseteq \{1, \ldots, D\}$,

$$(j_i, t_i) = \arg\min_{j \in S_i} \min_{t \in \mathcal{T}_j} \frac{|\mathcal{D}_i^L(j,t)|}{|\mathcal{D}_i|} c(\mathcal{D}_i^L(j,t)) + \frac{|\mathcal{D}_i^R(j,t)|}{|\mathcal{D}_i|} c(\mathcal{D}_i^R(j,t))$$

Ventaja frente a bagging: los bosques suelen ser más precisos que bagging pues muchas características son irrelevantes

Ejemplo: clasificación de correos en spam y no-spam

In [ ]:
```
import pandas as pd; import numpy as np
from sklearn.ensemble import RandomForestClassifier; from sklearn.metrics import accuracy_score
data_url = 'https://github.com/empathy87/The-Elements-of-Statistical-Learning-Python-Notebooks/blob/master/data/'
file = 'Spam.txt?raw=True'; df = pd.read_csv(data_url + file)
is_test = df.test.values; y = df.spam.values; X = df.drop(['test','spam'], axis=1).to_numpy(copy=True)
X_train, X_test = X[is_test == 0], X[is_test == 1]
y_train, y_test = y[is_test == 0], y[is_test == 1]
ntrees_list = [10, 100, 1000, 10000]
for ntrees in ntrees_list:
    clf = RandomForestClassifier(n_estimators=ntrees, random_state=10).fit(X_train, y_train)
    y_test_hat = clf.predict(X_test)
    acc = accuracy_score(y_test, y_test_hat)
    print(f'RF {ntrees} trees, test err {1 - acc:.1%}')
```

```
RF 10 trees, test err 6.3%
RF 100 trees, test err 4.9%
RF 1000 trees, test err 4.6%
RF 10000 trees, test err 4.8%
```

5 Boosting

Modelo aditivo de funciones base adaptativas: ensamble visto como suma de modelos base, no necesariamente árboles

$$f(\boldsymbol{x};\boldsymbol{\theta}) = \sum_{m=1}^{M} \beta_m \, F_m(\boldsymbol{x};\boldsymbol{\theta}_m)$$

Objetivo: minimizar la pérdida empírica (con regularizador)

$$\mathcal{L}(f) = \sum_{i=1}^{N} \ell(y_i, f(\boldsymbol{x}_i))$$

Boosting (potenciación): modelado aditivo por etapas hacia adelante o **forward stagewise additive modeling (FSAM)**

$$f_m(\boldsymbol{x}) = f_{m-1}(\boldsymbol{x}) + \beta_m F_m(\boldsymbol{x}) \quad \text{con} \quad F_m(\boldsymbol{x}) = F(\boldsymbol{x};\boldsymbol{\theta}_m)$$

donde el peso y parámetros del **weak learner (aprendiz)** añadido en la etapa m dependen del **strong learner** previo

$$(\beta_m, \boldsymbol{\theta}_m) = \operatorname*{argmin}_{\beta,\boldsymbol{\theta}} \; L_m(\beta, \boldsymbol{\theta})$$

con

$$L_m(\beta, \boldsymbol{\theta}) = \sum_{i=1}^{N} \ell(y_i, f_{m-1}(\boldsymbol{x}_i) + \beta F(\boldsymbol{x}_i;\boldsymbol{\theta}))$$

Ventaja frente a bagging y bosques: ofrece mejores resultados pues reduce el sesgo del aprendiz fuerte ajustando árboles que dependen unos de otros; bagging y bosques solo reducen la varianza ajustando árboles independientes

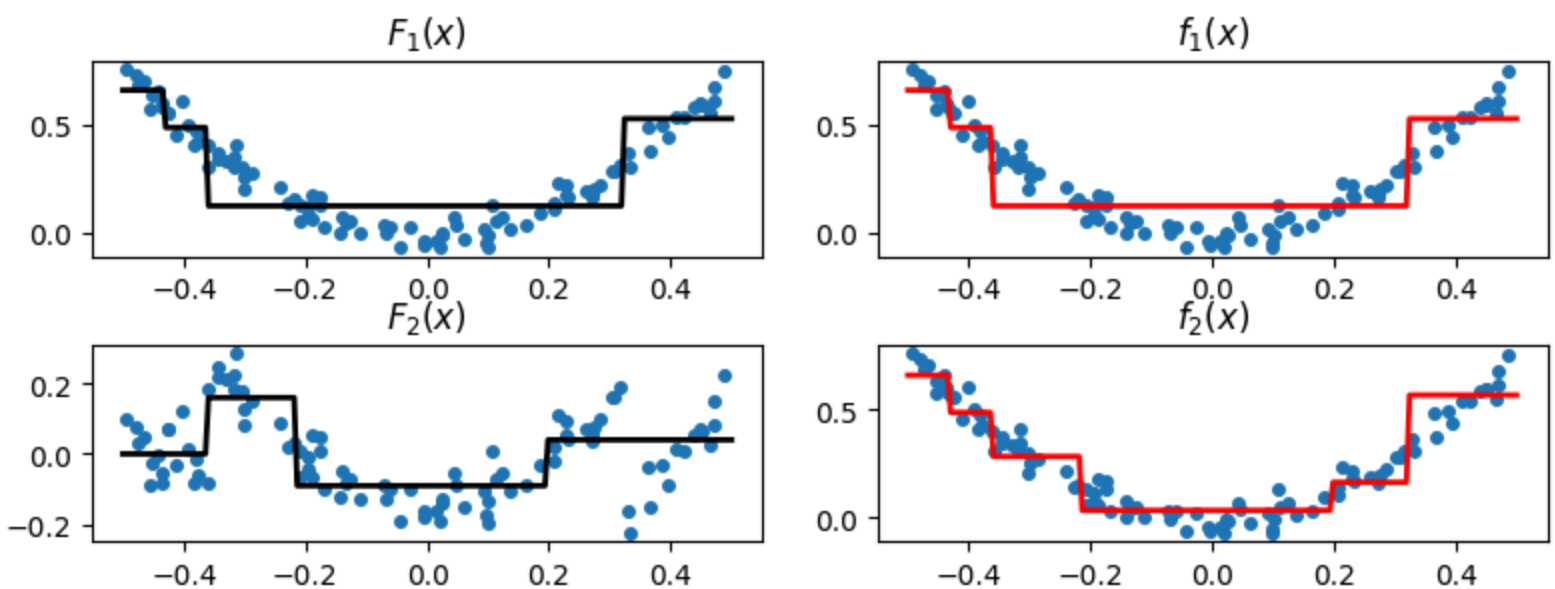
Objetivo FSAM con pérdida cuadrática: $\ell(y, \hat{y}) = (y - \hat{y})^2$

$$L_m(\beta, \boldsymbol{\theta}) = \sum_i (r_{im} - \beta F(\boldsymbol{x}_i; \boldsymbol{\theta}))^2 \qquad \text{con residuos} \qquad r_{im} = y_i - f_{m-1}(\boldsymbol{x}_i)$$

Boosting mínimos cuadrados: minimiza el objetivo fijando $\beta = 1$ y ajustando F a los residuos

Ejemplo: regresión simple con boosting mínimos cuadrados

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; from sklearn.tree import DecisionTreeRegressor
np.random.seed(42); X = np.random.rand(100, 1) - 0.5; y = 3 * X[:, 0] ** 2 + 0.05 * np.random.randn(100)
M = 2; fig, axes = plt.subplots(M, 2, figsize=(8, M * 1.5)); fig.tight_layout()
res = np.copy(y); tt = []; xx = np.linspace(-0.5, 0.5, 200)
for m in np.arange(M):
    tree = DecisionTreeRegressor(max_depth=2, random_state=42).fit(X, res); tt.append(tree)
    ax = axes[m, 0]; ax.set_title('$F_{}(x)$'.format(m+1)); ax.scatter(X, res, s=16)
    res_pred = tree.predict(xx.reshape(-1, 1)); ax.plot(xx, res_pred, 'k-', linewidth=2)
    ax = axes[m, 1]; ax.set_title('$f_{}(x)$'.format(m+1)); ax.scatter(X, y, s=16)
    y_pred = sum(t.predict(xx.reshape(-1, 1)) for t in tt)
    ax.plot(xx, y_pred, 'r-', linewidth=2); res -= tree.predict(X)
```



Objetivo FSAM con pérdida exponencial: $\ell(\tilde{y}, \hat{y}) = \exp(-\tilde{y}\hat{y})$ con $\tilde{y} \in \{-1, +1\}$

$$L_m(\beta, \boldsymbol{\theta}) = \sum\nolimits_i \exp(-\tilde{y}_i(f_{m-1}(\boldsymbol{x}_i) + \beta F(\boldsymbol{x}_i; \boldsymbol{\theta}))) = (e^{\beta} - e^{-\beta}) \sum\nolimits_i w_{im} \mathbb{I}(\tilde{y}_i \neq F(\boldsymbol{x}_i; \boldsymbol{\theta})) + e^{-\beta} \sum\nolimits_i w_{im}$$

Adaboost: primero halla $\boldsymbol{\theta}_m$ a partir de los datos ponderados, luego β_m y pesos para la iteración $m+1$

$$\boldsymbol{\theta}_m = \operatorname*{argmin}_{\boldsymbol{\theta}} \sum\nolimits_i w_{im} \mathbb{I}(\tilde{y}_i \neq F(\boldsymbol{x}_i; \boldsymbol{\theta})) \qquad \text{con} \quad w_{i1} = 1/N$$

$$\beta_m = \operatorname*{argmin}_{\beta} \; L_m(\beta, \boldsymbol{\theta}_m) = \frac{1}{2} \log \frac{1 - \mathrm{err}_m}{\mathrm{err}_m} \qquad \text{con} \quad \mathrm{err}_m = \frac{1}{\sum_i w_{im}} \sum\nolimits_i w_{im} \mathbb{I}(\tilde{y}_i \neq F_m(\boldsymbol{x}_i))$$

$$w_{i,m+1} = \begin{cases} w_{im} \exp(2\beta_m) & \text{si } \tilde{y}_i \neq F_m(\boldsymbol{x}_i) \\ w_{im} & \text{en otro caso} \end{cases}$$

Modelo ajustado para clasificación binaria: $f(\boldsymbol{x}) = \mathrm{sgn}(\sum_m \beta_m F_m(\boldsymbol{x}))$

Ejemplo: clasificación de correos en spam y no-spam

In [ ]:
```
import warnings; warnings.filterwarnings('ignore'); import pandas as pd; import numpy as np
from sklearn.ensemble import AdaBoostClassifier; from sklearn.metrics import accuracy_score
data_url = 'https://github.com/empathy87/The-Elements-of-Statistical-Learning-Python-Notebooks/blob/master/data/'
file = 'Spam.txt?raw=True'; df = pd.read_csv(data_url + file)
is_test = df.test.values; y = df.spam.values; X = df.drop(['test','spam'], axis=1).to_numpy(copy=True)
X_train, X_test = X[is_test == 0], X[is_test == 1]
y_train, y_test = y[is_test == 0], y[is_test == 1]
ntrees_list = [10, 100, 1000, 10000]
for ntrees in ntrees_list:
    clf = AdaBoostClassifier(n_estimators=ntrees, random_state=10, learning_rate=0.2).fit(X_train, y_train)
    acc = accuracy_score(y_test, clf.predict(X_test))
    print(f'AdaBoosting {ntrees} trees, test err {1 - acc:.1%}')
```

```
AdaBoosting 10 trees, test err 14.2%
AdaBoosting 100 trees, test err 7.5%
AdaBoosting 1000 trees, test err 6.3%
AdaBoosting 10000 trees, test err 5.2%
```

6 Gradient boosting

Gradient boosting: FSAM visto como descenso por gradiente para un problema de minimización en un espacio funcional

$$\hat{\boldsymbol{f}} = \operatorname{argmin}_{\boldsymbol{f}} \mathcal{L}(\boldsymbol{f}) \quad \text{con} \quad \mathcal{L}(\boldsymbol{f}) = \sum_{i=1}^{N} \ell(y_i, f(\boldsymbol{x}_i))$$

Extreme gradient boosting (XGBoost): implementación muy popular con algunos refinamientos

Ejemplo: clasificación de correos en spam y no-spam

In [ ]:
```
import warnings; warnings.filterwarnings('ignore'); import pandas as pd; import numpy as np
from xgboost import XGBClassifier; from sklearn.metrics import accuracy_score
data_url = 'https://github.com/empathy87/The-Elements-of-Statistical-Learning-Python-Notebooks/blob/master/data/'
file = 'Spam.txt?raw=True'; df = pd.read_csv(data_url + file)
is_test = df.test.values; y = df.spam.values; X = df.drop(['test','spam'], axis=1).to_numpy(copy=True)
X_train, X_test = X[is_test == 0], X[is_test == 1]
y_train, y_test = y[is_test == 0], y[is_test == 1]
ntrees_list = [10, 100, 1000, 10000]
for ntrees in ntrees_list:
    clf = XGBClassifier(n_estimators=ntrees, random_state=23).fit(X_train, y_train)
    acc = accuracy_score(y_test, clf.predict(X_test))
    print(f'XGBoosting {ntrees} trees, test err {1 - acc:.1%}')
```

```
XGBoosting 10 trees, test err 6.3%
XGBoosting 100 trees, test err 4.9%
XGBoosting 1000 trees, test err 4.8%
XGBoosting 10000 trees, test err 5.2%
```

7 Interpretación

Importancia de una característica k: suma de ganancias (reducciones de coste) promedio en los nodos que la usan

$$R_k = \frac{1}{M}\sum_{m=1}^{M} R_k(T_m) \qquad \text{con} \qquad R_k(T) = \sum_j G_j \mathbb{I}(v_j = k)$$

Ejemplo: importancias para clasificador MNIST (de dígitos escogidos)

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml; from sklearn.ensemble import RandomForestClassifier
mnist = fetch_openml('mnist_784', version=1, parser='auto')
mnist.data = mnist.data.astype(np.float32).to_numpy(); mnist.target = mnist.target.astype(np.uint8).to_numpy()
X = mnist["data"]; y = mnist["target"]; mask = (y == 6) | (y == 9) # <<< escoge dígitos
X_mask = X[mask, :]; y_mask = y[mask]; # print(X_mask.shape, y_mask.shape)
clf = RandomForestClassifier(n_estimators=20, random_state=42).fit(X_mask, y_mask)
image = clf.feature_importances_.reshape(28, 28)
_, ax = plt.subplots(1, 1, figsize=(3.5, 3.5)); ax.set(aspect='equal'); ax.axis('off')
plt.imshow(image, cmap='Blues_r', interpolation="none", vmin=0.000, vmax=0.001)
plt.colorbar(ax=ax, shrink=0.75);
```

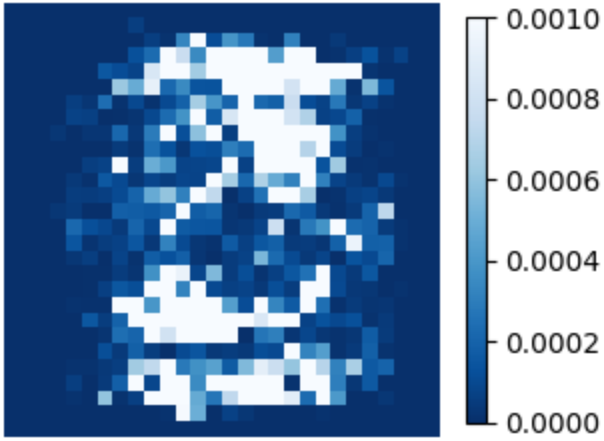
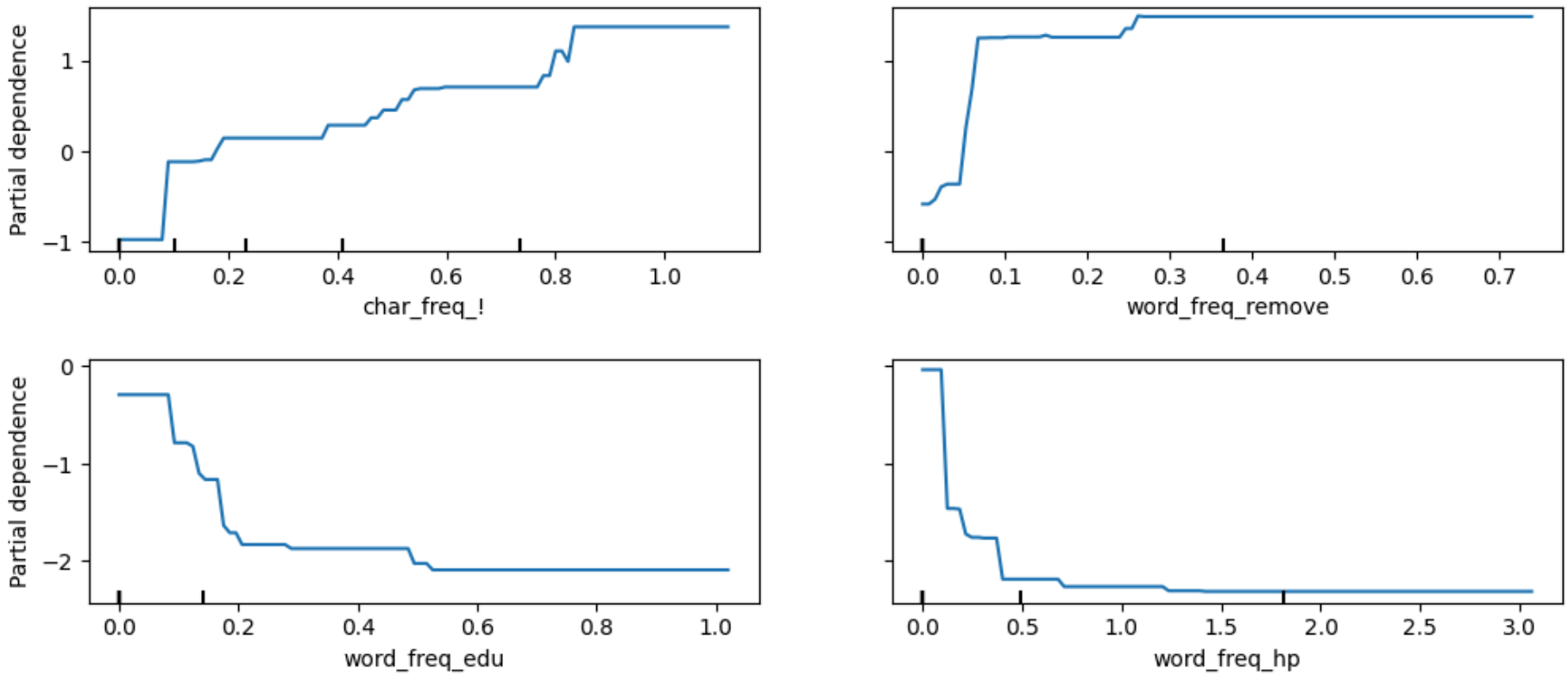


Gráfico de dependencia parcial: muestra la predicción del modelo en función de una (x_k) o dos (x_j, x_k) características

$$\bar{f}_k(x_k) = \frac{1}{N}\sum\nolimits_n f(\boldsymbol{x}_{n,-k}, x_k) \qquad \text{o} \qquad \bar{f}_{jk}(x_j, x_k) = \frac{1}{N}\sum\nolimits_n f(\boldsymbol{x}_{n,-jk}, x_j, x_k)$$

Ejemplo: logodds de spam; aumenta con la frecuencia de `!` y `remove`; `edu` y `hp` la disminuyen

In [ ]:
```
import pandas as pd; import matplotlib.pyplot as plt; from sklearn.ensemble import GradientBoostingClassifier
from sklearn.inspection import PartialDependenceDisplay as PDD
url = "https://github.com/empathy87/The-Elements-of-Statistical-Learning-Python-Notebooks/blob/master/data/"
df = pd.read_csv(url+"Spam.txt?raw=True"); X = df.drop(['test','spam'], axis=1); y = df.spam.values; #X[:15].T[:7]
clf = GradientBoostingClassifier(n_estimators=100, random_state=0).fit(X, y)
_, ax = plt.subplots(figsize=(12, 2)); PDD.from_estimator(clf, X, ['char_freq_!', 'word_freq_remove'], ax=ax)
_, ax = plt.subplots(figsize=(12, 2)); PDD.from_estimator(clf, X, ['word_freq_edu', 'word_freq_hp'], ax=ax);
```